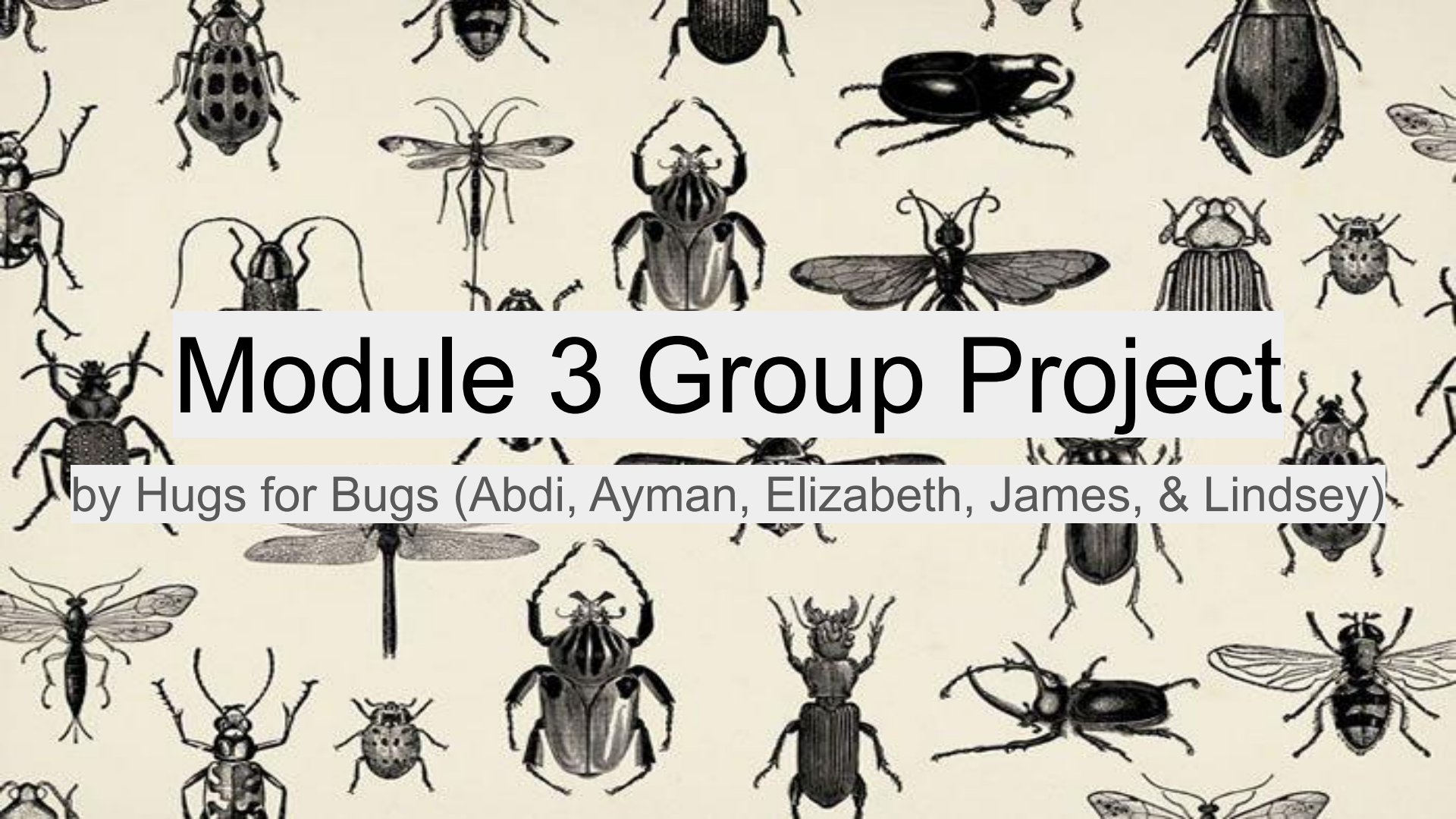




Module 3 Group Project

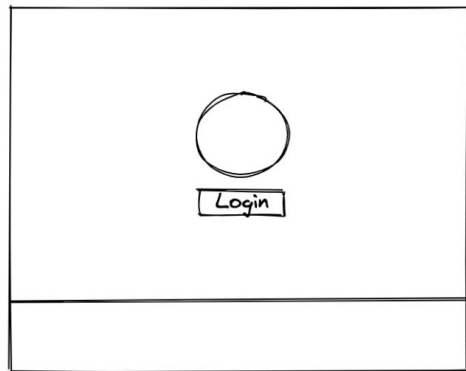
by Hugs for Bugs (Abdi, Ayman, Elizabeth, James, & Lindsey)

Proposed Update Stretch Goals

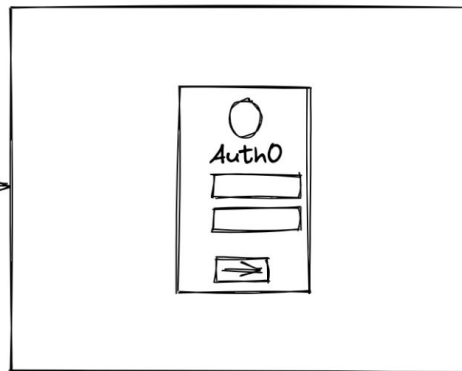
- We decided to integrate an Auth0 login screen to access warehouses
 - User Stories:
 - As a user I want to log in to the app to view warehouses.
 - As a user I want to click on an aisle to view the items as a list.
 - What Stakeholders might be involved?
 - Retail, warehouse, inventory and UX reps
 - What did your CI/CD flow look like at a high level?
- Individual warehouse login
 - Logout option
 - Shopping Cart
 - Timed logout

Wireframes

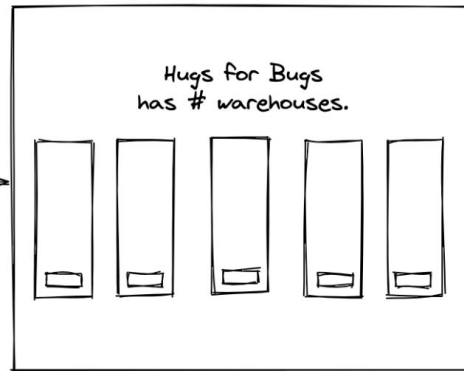
Landing



Login

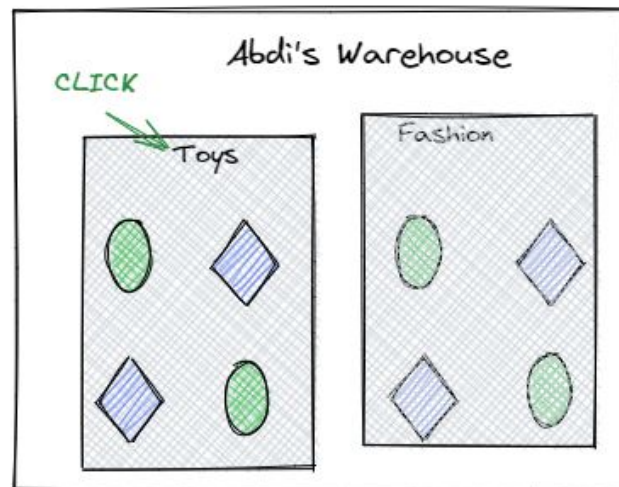


After login, redirects to /warehouses

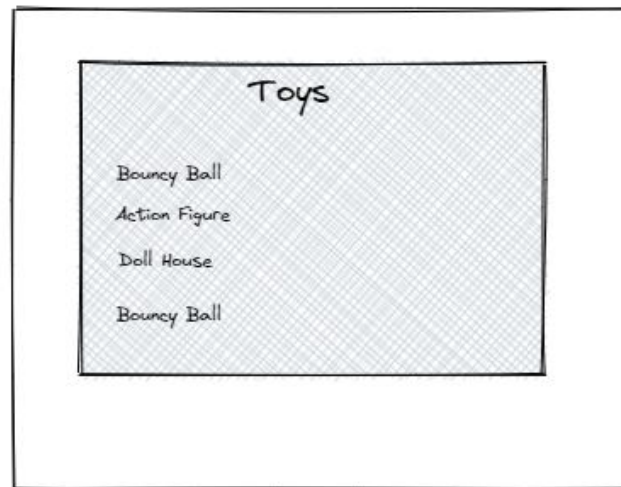


CLICK

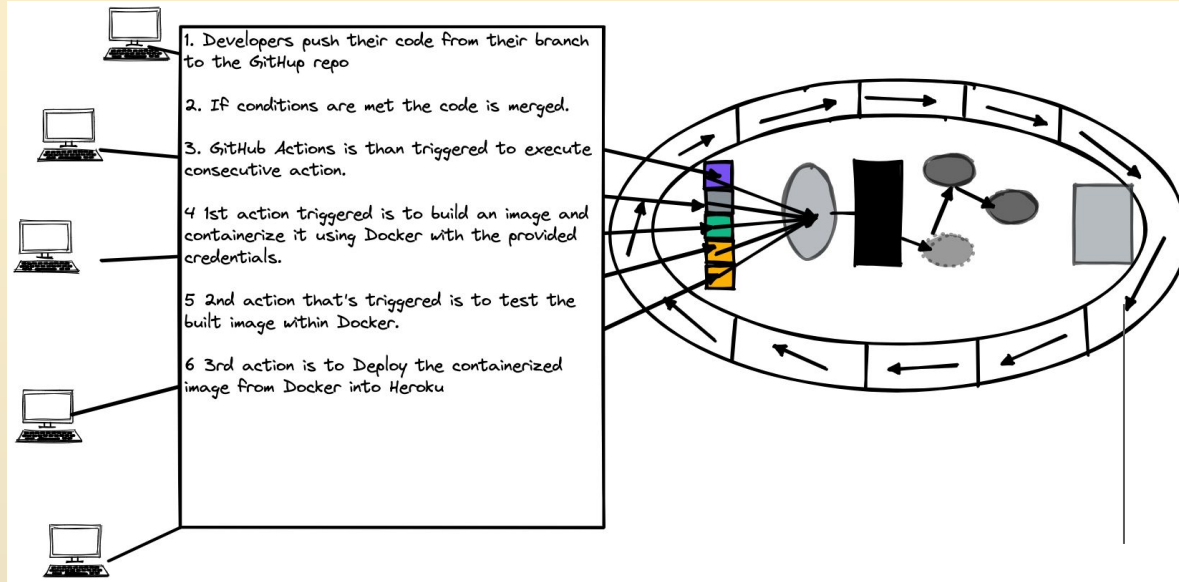
Abdi's Warehouse



Toys

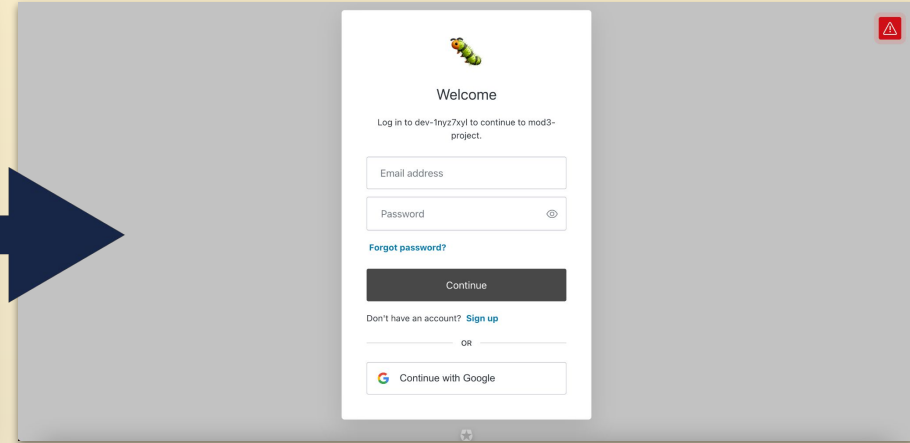


CI/CD Diagram



App Walkthrough

Deployed at <https://mod3-project.herokuapp.com/>



Development

- We added several dependencies along with new code to implement Auth0, along with auth.js & index.js files
- Our process involved a lot of mob programming and pushing changes to different branches - we also created a backup deployment of our app using an additional repo, in case of our authorization additions causing deployment issues

```
server.js
// Configure Passport to use Auth0
var strategy = new Auth0Strategy({
  domain: process.env.AUTH0_DOMAIN,
  clientId: process.env.AUTH0_CLIENT_ID,
  clientSecret: process.env.AUTH0_CLIENT_SECRET,
  callbackURL:
    process.env.AUTH0_CALLBACK_URL || 'https://mod3-project.herokuapp.com/warehouses'
});
function (accessToken, refreshToken, extraParams, profile, done) {
  return done(null, profile);
};
passport.use(strategy);
// You can use this section to keep a smaller payload
passport.serializeUser(function (user, done) {
  done(null, user);
});
passport.deserializeUser(function (user, done) {
  done(null, user);
});
const app = express();
app.get('/test', (req, res, next) => {
  res.send({ message: "Server is on" });
});
// View engine setup
app.use(logger('dev'));
app.use(cookieParser());

// config express-session
var sess = {
  secret: '3096b901406ab49275efc1f3229842526b4e9f7bbf22359b805b8ec6635aaed',
  cookie: {},
  resave: false,
  saveUninitialized: true
};
```

```
auth.js
var express = require('express');
var router = express.Router();
var passport = require('passport');
var dotenv = require('dotenv');
dotenv.config();
// Perform the login, after login Auth0 will redirect to callback
router.get('/login', passport.authenticate('auth0', {
  scope: 'openid email profile' }), function (req, res) {
  res.redirect('/warehouses');
});
module.exports = router;
```

```
const logger = require('morgan');
const cookieParser = require('cookie-parser');
const session = require('express-session');
const dotenv = require('dotenv');
const passport = require('passport');
const Auth0Strategy = require('passport-auth0');
const flash = require('connect-flash');
const path = require('path');
const authRouter = require('../routes/auth');
const indexRouter = require('../routes/index');
```

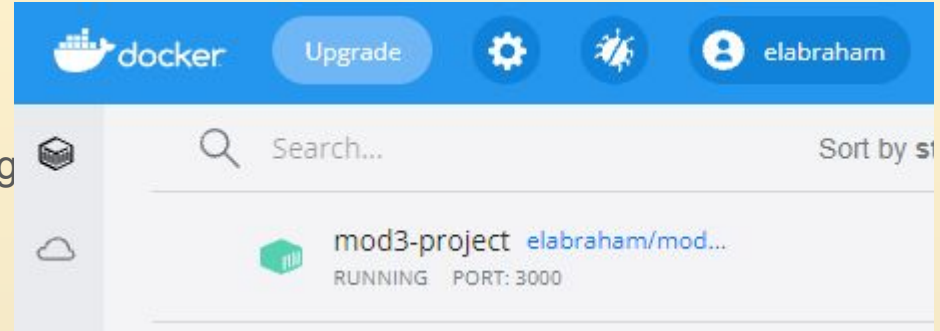
Testing

Manual integration testing locally before pushing to github branch

Built as a Docker Image and ran in a Docker Container to make sure the app could be built and run with newly integrated code

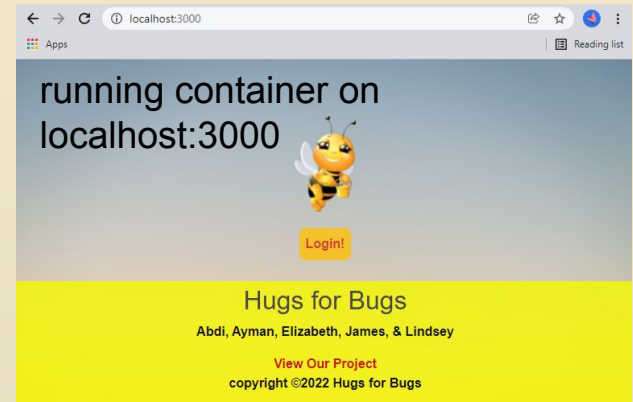
What would you do differently?

- Not have to seed locally before
- look into automating tests that make sure images seed and load correctly



```
$ docker build -t elabraham/mod3-project .
```

```
HP@DESKTOP-QL650ET MINGW64 ~/projects/multiverse/mod3-project (ea)  
$ docker run -d -p 3000:3000 --name=mod3-project elabraham/mod3-project
```



Deployment 🦋

- We utilized GitHub Actions to automate the creation of building a Docker image and deploying the app to Heroku after each push to our main branch
- With more time, we would have experimented more with automating additional jobs (like testing)

```
1 name: Deploy .github/workflow/main.yml
2
3 on:
4   push:
5     branches:
6       - main
7       - master
8
9 jobs:
10   build:
11     runs-on: ubuntu-latest
12     steps:
13       - uses: actions/checkout@v2
14       -
15         name: Set up Docker Buildx
16         uses: docker/setup-buildx-action@v1
17       -
18         name: Login to DockerHub
19         uses: docker/login-action@v1
20         with:
21           username: ${ secrets.DOCKERHUB_USERNAME }
22           password: ${ secrets.DOCKERHUB_TOKEN }
23       -
24         name: Build and push
25         id: docker_build
26         uses: docker/build-push-action@v2
27         with:
28           push: true
29           tags: ${ secrets.DOCKERHUB_USERNAME }/mod-3:latest #must m
30
31       - uses: akhileshns/heroku-deploy@v3.12.12 # This is the action
32         with:
33           heroku_api_key: ${ secrets.HEROKU_API_KEY }
34           heroku_app_name: "mod3-project" #Must be unique in Heroku
35           heroku_email: "lndsadams@gmail.com"
36           usedocker: true
```

FROM `node:lts-alpine`

WORKDIR `/docker`

COPY `["package*.json", "./*"]`

RUN `npm install`

COPY `. .`

CMD `["npm", "start"]`

Dockerfile

Actions secrets

New repository secret

Secrets are environment variables that are encrypted. Anyone with collaborator access to this repository can use these secrets for Actions.

Secrets are not passed to workflows that are triggered by a pull request from a fork. [Learn more.](#)

🔒 DOCKERHUB_TOKEN	Updated yesterday	Update	Remove
🔒 DOCKERHUB_USERNAME	Updated yesterday	Update	Remove
🔒 HEROKU_API_KEY	Updated yesterday	Update	Remove

build

succeeded yesterday in 1m 11s

🔍 Search logs



> ✓ Set up job	1s
> ✓ Run actions/checkout@v2	1s
> ✓ Set up Docker Buildx	3s
> ✓ Login to DockerHub	1s
> ✓ Build and push	21s
> ✓ Run akhileshns/heroku-deploy@v3.12.12	43s
> ✓ Post Build and push	0s
> ✓ Post Login to DockerHub	0s
> ✓ Post Set up Docker Buildx	1s
> ✓ Post Run actions/checkout@v2	0s
> ✓ Complete job	0s

THE END. Thank you!